JOURNAL OF COMPUTATIONAL BIOLOGY
Volume 25, Number 0, 2018

Pp. 1–13
DOI: 10.1089/cmb.2017.0257

Research Article

A Simple Linear Space Algorithm for Computing Nonoverlapping Inversion and Transposition Distance in Quadratic Average Time

XIAODONG WANG[1] and LEI WANG[2]

ABSTRACT

In the sequence alignment problem, it is important to compare DNA sequences to retrieve relevant information and align these sequences. An inversion and a translocation are important operations in comparing DNA sequences in biosequence analysis. The alignment problem with nonoverlapping inversions and translocations is to find an alignment with nonoverlapping inversions and translocations for the given two strings *X* and *Y*. This problem has interesting application for finding a common sequence from two mutated sequences. A linear space and quadratic average time algorithm to compute the mutation distance between two strings of the same length under nonoverlapping inversions and transpositions is presented in this article. The recursive formula for this purpose is novel to the best of our knowledge. The space costs of the algorithms to solve the same problem are typically quadratic, and thus, our original algorithm is the first linear space algorithm to solve the mutation distance problem.

Keywords: Mutation distance, time and space costs, quadric average time, linear space algorithm.

1. INTRODUCTION

In modern computational biology, it is important to compare DNA sequences to retrieve relevant information and align these sequences (Schoniger and Waterman, 1992; Sakai, 2011; Cantone et al., 2013). Alignments are usually represented as edit sequences that transform one sequence into the other under operations that model the evolutionary process. Genetic mutations can cause many hereditary diseases. It has been known for a long time that a chromosomal inversion occurs when a single chromosome undergoes breakage and rearrangement within itself (Schoniger and Waterman, 1992). A chromosomal translocation relocates a piece of the DNA sequence from one place to another. Therefore, the chromosomal inversion and translocation operations can alter a DNA sequence and cause genetic diseases, and thus they are crucial in alignments (Oliver-Bonet and Navarro, 2002; Li and Ng, 2009).

The problem of string matching with inversion and translocation is related to the alignment problem with nonoverlapping inversions and translocations. It is also the main topic of this article. The problem of string matching with inversion and translocation is to find all matching of a given pattern P in a text T allowing

[1]Fujian University of Technology, Fuzhou, China.
[2]Facebook, Inc., Menlo Park, California.

inversion and translocation. The alignment problem with inversion and translocation is to transform a given string X to another string Y allowing inversion and translocation edit operations. For two strings X and Y of the same length n, the nonoverlapping inversion and the transposition distance (also called mutation distance) between them are defined as the minimum number of nonoverlapping inversion and transposition operations used to transform one string into the other. Many researchers investigated efficient algorithms for the pattern matching and alignment problem with inversion and translocation, and many efficient algorithms have been developed for pattern matching and alignment problem with inversion and translocation (Gao et al., 2003; Chen and Gao, 2004; Vellozo et al., 2006; Lipsky et al., 2007; Grabowski et al., 2011; Apostolico et al., 2016).

Cho et al. (2015) have considered the alignment problem in the case of nonoverlapping inversion and translocation allowed for two strings X and Y. They presented an efficient algorithm to determine the existence of such an alignment and retrieve an alignment (Huang and Lu, 2010; Huang et al., 2014; Cho et al., 2015). In recent research, Ta et al. (2016) presented an $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ space algorithm to compute the mutation distance of two input strings.

In this article, we are also interested in the computation of the mutation distance between two strings of the same length under nonoverlapping inversions and transpositions, where the lengths of two adjacent and nonoverlapping fragments exchanged by a transposition can be different. We put forward a very simple linear space algorithm to solve the mutation distance problem in quadratic average time.

The organization of the article is as follows.

In the following five sections, we describe our new algorithm for computing nonoverlapping inversion and transposition distance.

In Section 2, the preliminary knowledge for presenting our new algorithm is discussed. In Section 3, we present a very simple dynamic programming algorithm to compute the mutation distance of two input strings of the same length n, using $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ space. In Section 4, the time cost of the new algorithm is reduced to average $\mathcal{O}(n^2)$. In Section 5, the space cost of the new algorithm is reduced to $\mathcal{O}(n)$. Some concluding remarks are placed in Section 6.

2. PRELIMINARIES

In the whole article, we will use $X = x_1x_2 \cdots x_n$ and $Y = y_1y_2 \cdots y_n$ to denote the two input strings of size n over a finite alphabet $\sum$. In the following discussion, the only alphabet of interest is the DNA alphabet $\sum = \{A, C, G, T\}$.

For the given string X of length n, the ith character of X is denoted as $x_i \in \sum$, for any $i = 1, \cdots, n$. A *substring* of X from position i to j is denoted as $X[i, j] = x_ix_{i+1} \cdots x_j$, for $1 \leq i \leq j \leq n$.

If $i \neq 1$ or $j \neq n$, then the substring $X[i, j]$ is called a *proper substring* of X. A substring $X[i, j]$ is called a *prefix* or a *suffix* of X if $i = 1$ or $j = n$, respectively. For any prefix $X[1, i]$ of X, the substring $X[k, i]$, $1 \leq k \leq i$, is also called the *suffix* of $X[1, i]$ *starting at position k*. For two prefixes $X[1, i]$ of X and $Y[1, j]$ of Y, if a substring S is a suffix of both $X[1, i]$ and $Y[1, j]$, then S is called a *common suffix* of $X[1, i]$ and $Y[1, j]$. The *longest common suffix of prefixes* $X[1, i]$ and $Y[1, j]$ is a common suffix of maximal length. The *length of the longest common suffix* of $X[1, i]$ and $Y[1, j]$ is denoted by $lcs(i, j, X, Y)$. It is obvious that $lcs(i, j, X, Y)$ can be computed recursively as follows:

$$lcs(i, j, X, Y) = \begin{cases} 1 + lcs(i-1, j-1, X, Y) & \text{if } x_i = y_j; \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

In DNA sequences, $\sum = \{A, C, G, T\}$, and A, T and C, G are *complementary base pairs*.

Definition 1.

For each character $c \in \sum$, $\theta(c)$ is its complementary character in $\sum$.
For the DNA alphabet, $\theta(A) = T$, $\theta(T) = A$, $\theta(G) = C$, and $\theta(C) = G$.
For any substring $X[i, j]$ of X, the inversion operation θ acting on $X[i, j]$ is the string

$$\theta(X[i, j]) = \theta(x_j)\theta(x_{j-1}) \cdots \theta(x_i). \quad (2)$$

$\theta(X[i, j])$ is also denoted as $\theta_{i,j}(X)$.

Definition 2.

For example, if $X = TAGGAC$, *then* $\theta_{2,4}(X) = TCCTAC$.

The transposition operation τ *acting on two nonempty strings U and V is* $\tau(UV) = VU$ *The lengths of U and V can be different.*

For any substring $X[i, j]$ *of X, and an integer k such that* $1 \leq i < k \leq j \leq n$, *the transposition operation* τ *acting on* $X[i, j]$ *and k is the string*

$$\tau_{i,j,k}(X) = X[k, j]X[i, k-1]. \tag{3}$$

Definition 3.

For example, if $X = TAGGAC$, *then* $\tau_{2,5,4}(X) = TGAAGC$.

The operations θ *and* τ *defined above are called mutation operations. For the mutation operations* $\theta_{i,j}$ *and* $\tau_{i,j,k}$, *the index range* $[i, j]$ *is called their mutation range, and for* $t \in [i, j]$, *the operations* $\theta_{i,j}$ *and* $\tau_{i,j,k}$ *cover t. For any two mutation operations, if the intersection of their mutation ranges is empty, then they are nonoverlapping mutation operations.*

Definition 4.

For two strings X and Y of the same length, the minimum number of pairwise nonoverlapping mutation operations used to transform X into Y, denoted by $md(X, Y)$, *is called the mutation distance between X and Y. If there is no set of nonoverlapping mutation operations that can convert X into Y, then* $md(X, Y)$ *is infinity.*

Definition 5.

If a set Θ *of nonoverlapping mutation operations consecutively applied on the string X can convert X into Y, then the set* Θ *is called a feasible mutation operation set, and is denoted as* $\Theta(X) = Y$. *It follows that if* $\theta_{i,j} \in \Theta$, *then* $\theta_{i,j}(X) = Y[i, j]$. *Similarly, if* $\tau_{i,j,k} \in \Theta$, *then* $\tau_{i,j,k}(X) = Y[i, j]$. *The mutation operations in a feasible mutation operation set* Θ *are called feasible mutation operations. The number of mutation operations in* Θ *is denoted by* $|\Theta|$. *In the case of* $|\Theta| = md(X, Y)$, *the set* Θ *is an optimal mutation operation set of X and Y.* $Md(X, Y)$ *is the set of optimal mutation operation sets of X and Y:*

$$Md(X, Y) = \{\Theta | \Theta \text{ is an optimal mutation operation set of } X \text{ and } Y\}. \tag{4}$$

Example.

Let $\sum = \{A, C, G, T\}$, $X = TAGGAC$ *and* $Y = TAGACG$.

There are only four feasible mutation operation sets $\Theta_1 = \{\theta_{1,2}, \tau_{3,6,4}\}$, $\Theta_2 = \{\theta_{1,2}, \tau_{4,6,5}\}$, $\Theta_3 = \{\tau_{3,6,4}\}, \Theta_4 = \{\tau_{4,6,5}\}$. *It is readily seen that* $|\Theta_1| = |\Theta_2| = 2$ *and* $|\Theta_3| = |\Theta_4| = 1$. *It follows that* $md(X, Y) = 1$ *and* $Md(X, Y) = \{\Theta_3, \Theta_4\}$.

Definition 6.

For each $1 \leq i \leq j \leq n$, *let*

$$\xi(i, j) = \begin{cases} i & \text{if } \theta_{i,j}(X) = Y[i, j]; \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

$$\eta(i, j) = \begin{cases} k & \text{if there exists } i < k \leq j \text{ such that } \tau_{i,j,k}(X) = Y[i, j]; \\ 0 & \text{otherwise.} \end{cases} \tag{6}$$

By the definition, $\xi(i, j)$ is the start position i such that $\theta_{i,j}$ can be applied to produce $Y[i, j]$, otherwise $\xi(i, j)$ is set to 0. Similarly, if there exists $i < k \leq j$ such that $\tau_{i,j,k}$ can be applied to produce $Y[i, j]$, then the value of $\eta(i, j)$ is k. If no such k exists, $\eta(i, j)$ is set to 0.

It can be seen from the following Lemma 1 that if for each pair (i, j), $lcs(i, j, X, Y)$ can be computed in $\mathcal{O}(1)$ time, then $\xi(i, j)$ can be computed in $\mathcal{O}(1)$ time, while $\eta(i, j)$ may cost $\mathcal{O}(j - i)$ time in the worst case. In the following sections, string Z denotes the inversion of X.

$$Z = \theta(X) = \theta(x_n)\theta(x_{n-1}) \cdots \theta(x_1) \tag{7}$$

Lemma 1.
For each $1 \le i < j \le n$,

- $\xi(i, j) > 0$ *if and only if* $lcs(n-i+1, j, Z, Y) \ge j-i+1$.
- $\eta(i, j) = k > 0$ *if and only if*

$$\begin{cases} lcs(i+j-k, j, X, Y) \ge j-k+1 \\ lcs(j, k-1, X, Y) \ge k-i \end{cases} \tag{8}$$

Proof.
(1). It follows from Equations (2) and (7) that

$$\theta_{i,j}(X) = \theta(x_j)\theta(x_{j-1}) \cdots \theta(x_i) = Z[n-j+1, n-i+1]$$

and thus $\xi(i, j) > 0$ if and only if $\theta_{i,j}(X) = Y[i, j]$. This is equivalent to $Z[n-j+1, n-i+1] = Y[i, j]$. It follows that $\xi(i, j) > 0$ if and only if $lcs(n-i+1, j, Z, Y) \ge j-i+1$.

(2). It follows from Equation (3) that $\eta(i, j) = k > 0$ if and only if $\tau_{i,j,k}(X) = Y[i, j]$. This is equivalent to $X[k, j]X[i, k-1] = Y[i, j]$.

It follows that $\eta(i, j) = k > 0$ if and only if $\begin{cases} lcs(i+j-k, j, X, Y) \ge j-k+1 \\ lcs(j, k-1, X, Y) \ge k-i \end{cases}$.

The proof is completed. ■

3. THE DESCRIPTION OF THE ALGORITHM

Let X and Y be the given two input strings, and for $0 \le i \le n$, the mutation distance between $X[1, i]$ and $Y[1, i]$ be denoted by $f(i) = md(X[1, i], Y[1, i])$. Then, our problem is to find $f(n) = md(X, Y)$.

We now discuss how to find $f(i)$, $1 \le i \le n$ iteratively. There are three cases to be distinguished.

In the first case, let $f(i-1) < \infty$ and $x_i = y_i$. There must be a set Θ_{i-1} of nonoverlapping mutation operations such that $f(i-1) = |\Theta_{i-1}|$, and Θ_{i-1} can convert $X[1, i-1]$ into $Y[1, i-1]$. It follows from $x_i = y_i$ that Θ_{i-1} can also convert $X[1, i]$ into $Y[1, i]$. It follows that in this case $f(i) \le f(i-1)$.

In the second case, let $f(j-1) < \infty$, $1 \le j \le i$, and $\theta_{j,i}$ is a feasible inversion operation, that is, $\xi(j, i) > 0$. In this case, there must be a set Θ_{j-1} of nonoverlapping mutation operations such that $f(j-1) = |\Theta_{j-1}|$, and Θ_{j-1} can convert $X[1, j-1]$ into $Y[1, j-1]$. It follows from $\xi(j, i) > 0$ that $\Theta_{j-1} \bigcup \theta_{j,i}$ can convert $X[1, i]$ into $Y[1, i]$, and $|\Theta_{j-1} \bigcup \theta_{j,i}| = f(j-1)+1 < \infty$. It follows that $\Theta_{j-1} \bigcup \theta_{j,i}$ must be a candidate of Θ_i, and thus $f(i) \le f(j-1)+1$.

Similarly, in the third case, let $f(j-1) < \infty$, $1 \le j \le i$, and $\tau_{j,i,k}$ is a feasible transposition operation, that is, $k = \eta(j, i) > 0$. In this case, there must be a set Θ_{j-1} of nonoverlapping mutation operations such that $f(j-1) = |\Theta_{j-1}|$, and Θ_{j-1} can convert $X[1, j-1]$ into $Y[1, j-1]$. It follows from $k = \eta(j, i) > 0$ that $\Theta_{j-1} \bigcup \tau_{j,i,k}$ can convert $X[1, i]$ into $Y[1, i]$, and $|\Theta_{j-1} \bigcup \tau_{j,i,k}| = f(j-1)+1 < \infty$. It follows that $\Theta_{j-1} \bigcup \tau_{j,i,k}$ must be a candidate of Θ_i, and thus $f(i) \le f(j-1)+1$.

The three values $\alpha(i)$, $\beta(i)$, and $\gamma(i)$ corresponding to the three cases discussed above can be defined as follows.

$$\alpha(i) = \begin{cases} f(i-1) & \text{if } x_i = y_i \\ \infty & \text{Otherwise} \end{cases} \tag{9}$$

$$\beta(i) = \min_{1 \le j \le i,\ \xi(j,i) > 0} f(j-1)+1 \tag{10}$$

$$\gamma(i) = \min_{1 \le j \le i,\ \eta(j,i) > 0} f(j-1)+1 \tag{11}$$

In the following theorem, we can show a new recursive formula to compute $f(i)$ iteratively for all $1 \le i \le n$. The proof of the theorem is given in Appendix A.

Theorem 1. *For each* $1 \le i \le n$, $f(i) = \min\{\alpha(i), \beta(i), \gamma(i)\}$.

Based on Theorem 1, the mutation distance between the given input strings, $X = x_1x_2 \cdots x_n$ and $Y = y_1y_2 \cdots y_n$ of the same length n, can be computed by a standard dynamic programming algorithm as shown in Algorithm 1.

Algorithm 1: $md(X, Y)$

Input: Two strings X, Y of the same length n.
Output: The mutation distance $md(X, Y)$.

```
1  f(0) ← 0;
2  for i = 1 to n do
3  |   f(i) ← ∞; β ← ∞; γ ← ∞;
4  |   if x_i = y_i then f(i) ← f(i − 1);
5  |   for j = 1 to i do
6  |   |   if β > f(j − 1) and ξ(j, i) > 0 then
7  |   |   |   β ← f(j − 1); k ← η(j, i);
8  |   |   |   c(0) ← j;
9  |   |   end
10 |   |   If γ > f(j − 1) and k > 0 then
11 |   |   |   γ ← f(j − 1); d(0) ← k; e(0) ← j;
12 |   |   end
13 |   end
14 |   if f(i) > β + 1 then
15 |   |   f(i) ← β + 1; c(i) ← c(0);
16 |   end
17 |   if f(i) > γ + 1 then
18 |   |   f(i) ← γ + 1; d(i) ← d(0); e(i) ← e(0);
19 |   end
20 end
21 return f(n)
```

In the algorithm above, we can use two tables $a(j, i)$ and $b(j, i)$ to store the values of $lcs(j, i, X, Y)$ and $lcs(j, i, Z, Y)$, respectively, and then, the values of $\xi(j, i)$ and $\eta(j, i)$ can be computed efficiently. The two tables a and b can be precomputed in $\mathcal{O}(n^2)$ time as follows.

Algorithm 2: Preprocessing

```
1 for i = 1 to n do
2 |   for j = 1 to n do
3 |   |   a(i, j) ← 0, b(i, j) ← 0;
4 |   |   if x_i = y_j then a(i, j) ← a(i − 1, j − 1) + 1;
5 |   |   if z_i = y_j then b(i, j) ← b(i − 1, j − 1) + 1;
6 |   end
7 end
```

Then, $\xi(j, i)$ and $\eta(j, i)$ can be computed efficiently in $\mathcal{O}(1)$ time by Lemma 1.

Theorem 2.
Algorithm 1 computes the mutation distance of two strings of equal-length n in $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ space.

Proof.
In Algorithm 1, $\alpha(i)$ is computed in line 4, $\beta(i)$ and $\gamma(i)$ are computed in lines 6–8. Then, $f(i)$ is computed by Theorem 1 in lines 10–11. The correctness of Algorithm 1 can be verified according to Theorem 1 and the discussion mentioned above.

It follows from Lemma 1 that for each pair (j, i) in Algorithm 1, $\xi(j, i)$ can be computed in $\mathcal{O}(1)$ time, while $\eta(j, i)$ may cost $\mathcal{O}(i - j)$ time in the worst case. It follows that the time complexity of Algorithm 1 is $\mathcal{O}(n^3)$ in the worst case.

The space complexity of the algorithm is clearly $\mathcal{O}(n^2)$, since only two two-dimensional arrays of size $\mathcal{O}(n^2)$ are used in the algorithm.

The proof is completed. ■

In Algorithm 1, three one-dimensional tables c, d and e are used to store the information on the feasible mutation operations found by the algorithm, which can be used to rebuild an optimal mutation operation set in $Md(X, Y)$ at the end of Algorithm 1. With the information stored in the tables c, d and e, an optimal mutation operation set in $Md(X, Y)$ can be constructed by a recursive algorithm *Backtrack* as follows.

Algorithm 3: *Backtrack*(i)

```
1 if i < 1 then return;
2 l←c(i), k←d(i), j←e(i);
3 if k > 0 then
4 |  Backtrack(j − 1);
5 |  print τ(j, i, k);
6 end
7 else if l > 0 then
8 |  Backtrack(l − 1);
9 |  print θ(l, i);
10 end
11 else Backtrack(i − 1);
12 return;
```

A call *Backtrack*(n) will produce an optimal mutation operation set in $Md(X, Y)$ in $\mathcal{O}(n)$ time.

It seems that the time and space complexities of Algorithm 1 are similar to the algorithm of Cho et al. in Refs. Huang and Lu (2010), Huang et al. (2014), Cho et al., (2015), and the algorithm Ta et al. (2016). The reasons why Algorithm 1 is valuable and important are as follows. First, Algorithm 1 is a totally different algorithm from the algorithms for computing the mutation distance of two strings in the literature. Its novel recursive structure is very simple and easy to figure out as described above. More importantly, a very efficient linear space and quadratic average time algorithm to compute the mutation distance between two strings under nonoverlapping inversions and transpositions is drawn from Algorithm 1 in the following sections.

Example (*continued*).

Let the two given input strings be $X = TAGGAC$ and $Y = TAGACG$. $Z = \theta(X) = GTCCTA$.

The two tables $a(j, i)$ and $b(j, i)$ to store the values of $lcs(j, i, X, Y)$ and $lcs(j, i, Z, Y)$, respectively, can be computed as follows:

$$a = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \end{pmatrix}, b = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 1 & 0 & 0 \end{pmatrix}$$

The two tables $\xi(j, i)$ and $\eta(j, i)$, $1 \le j < i \le n$ can then be computed as follows:

$$\xi = \begin{pmatrix} 0 & & & & & \\ 1 & 0 & & & & \\ 0 & 0 & 0 & & & \\ 0 & 0 & 0 & 0 & & \\ 0 & 0 & 0 & 0 & 0 & \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \eta = \begin{pmatrix} 0 & & & & & \\ 0 & 0 & & & & \\ 0 & 0 & 0 & & & \\ 0 & 0 & 0 & 0 & & \\ 0 & 0 & 0 & 0 & 0 & \\ 0 & 0 & 4 & 5 & 0 & 0 \end{pmatrix}$$

$f[i]$, the mutation distance between $X[1, i]$ and $Y[1, i]$, $1 \le i \le n$, can then be computed by Algorithm 1.

$$f = (\, 0, 0, 0, \infty, \infty, 1 \,)$$

The three one-dimensional tables c, d and e to store the information on the feasible mutation operations are also computed.

$$c=(0,0,0,0,0,0),\ d=(0,0,0,0,0,4),\ e=(0,0,0,0,0,3)$$

Therefore, $f(n)=md(X,Y)=1$. Finally, a call $Backtrack(n)$ produces an optimal mutation operation set $\{\tau_{3,6,4}\}\in Md(X,Y)$.

4. REDUCE THE TIME COST TO AVERAGE $\mathcal{O}(n^2)$

The most time-consuming part of the new algorithm is to compute $\gamma(i)$ in lines 7–8, where $\eta(j,i)$ must be computed for each pair (j,i), $1\le j<i\le n$. In this section, we can speed up the computation of $\gamma(i)$ by making use of an efficient filter algorithm. The idea behind the filter algorithm is straightforward. It is based on the observation that if $\eta(j,i)>0$, then $X[j,i]$ must be a permutation of $Y[j,i]$.

Let $\sum=\{c_1,c_2,\cdots,c_\sigma\}$ be a finite alphabet, where $\sigma\ge 3$ is a constant. For each character $c\in\Sigma$, denote $ind(c)=k$ if $c=c_k$.

Definition 7.

For the two input strings $X=x_1x_2\cdots x_n$ and $Y=y_1y_2\cdots y_n$ over $\sum$, and each $c_k\in\sum$, $1\le k\le\sigma$, let

$$P(i)=\{j|1\le j\le i, X[j,i]\ \textit{is a permutation of}\ Y[j,i]\} \tag{12}$$

$$Q(i)=\{j|1\le j\le i, f(j-1)+1<f(i)\} \tag{13}$$

To compute $\gamma(i)$, only those pairs (j,i) such that $j\in P(i)\bigcap Q(i)$ need to be checked for $\eta(j,i)>0$. Other pairs can be filtered out. To quickly find $j\in P(i)$, we need some more notations. We define $o_x(i,k)=1$ if $x_i=c_k$, otherwise $o_x(i,k)=0$. The definition of $o_y(i,k)$ is similar. Then, the number of occurrences of character $\boldsymbol{c_k}$ in $X[j,i]$ and $Y[j,i]$, denoted by $g_x(j,i,k)$ and $g_y(j,i,k)$, can be defined as follows,

$$\begin{cases} g_x(j,i,k)=\sum_{t=j}^{i} o_x(t,k) \\ g_y(j,i,k)=\sum_{t=j}^{i} o_y(t,k) \end{cases} \tag{14}$$

The difference of $g_x(j,i,k)$ and $g_y(j,i,k)$ is denoted as $g(j,i,k)=g_x(j,i,k)-g_y(j,i,k)$.

The sum of absolute differences for all characters in $X[1,i]$ and $Y[1,i]$, is denoted as:

$$h(j,i)=\sum_{k=1}^{\sigma}|g(j,i,k)| \tag{15}$$

It follows that $h(j,i)=0$ is a necessary and sufficient condition of $j\in P(i)$.

The following lemma enables us to compute $h(j,i)$ and thus find $j\in P(i)$ efficiently.

Lemma 2.

$$h(j-1,i)=\begin{cases} h(j,i) & \textit{if}\ x_{j-1}=y_{j-1} \\ h(j,i)-\delta_1+\delta_2 & \textit{Otherwise} \end{cases} \tag{16}$$

where

$$\begin{cases} \delta_1=|g(j,i,ind(x_{j-1}))|+|g(j,i,ind(y_{j-1}))| \\ \delta_2=|g(j,i,ind(x_{j-1}))+1|+|g(j,i,ind(y_{j-1}))-1| \end{cases} \tag{17}$$

The proof of Lemma 2 is placed in Appendix B. Based on Lemma 2, a very efficient algorithm to compute $\gamma(i)$, for each $1\le i\le n$, can be described as follows.

Algorithm 4: $\gamma(i)$

Input: i, $1 \leq i \leq n$
1 **for** $j=1$ **to** i **do** $g(ind(x_j)) \leftarrow g(ind(y_j)) \leftarrow 0$;
2 $h \leftarrow 0$;
3 **for** $j=i$ **downto** 1 **do**
4 $\quad a \leftarrow ind(x_j)$, $b \leftarrow ind(y_j)$;
5 $\quad \delta_1 \leftarrow |g(a)| + |g(b)|$;
6 $\quad g(a) \leftarrow g(a)+1$, $g(b) \leftarrow g(b)-1$;
7 $\quad \delta_2 \leftarrow |g(a)| + |g(b)|$;
8 $\quad h \leftarrow h - \delta_1 + \delta_2$;
9 $\quad$ **if** $h=0$ **and** $f(i) > f(j-1)-1$ **and** $\eta(j,i) > 0$ **then** $f(i) \leftarrow f(j-1)-1$;
10 **end**

In the algorithm above, an array g is used to store the values of $g(j, i, c)$ in Lemma 2 for each character $c \in \Sigma$. In line 9, $h=0$ implies $j \in P(i)$ and $f(i) > f(j-1)-1$ implies $j \in Q(i)$, according to Lemma 2. Therefore, only those pairs (j, i) such that $j \in P(i) \bigcap Q(i)$ are checked for $\eta(j, i) > 0$. The time cost is then reduced substantially.

Theorem 3.

If we assume a uniform distribution and independence of characters, then the average time cost to compute $\gamma(i)$, $1 \leq i \leq n$, can be bounded above by $\mathcal{O}(n^2)$.

The proof of Theorem 3 is given in Appendix C.

The time cost of the other parts of Algorithm 1 is $\mathcal{O}(n^2)$. Therefore, if the filter algorithm is applied, the time cost of Algorithm 1 can be reduced to $\mathcal{O}(n^2)$ in average, according to Theorem 3. The space cost of Algorithm 4 is clearly $\mathcal{O}(\sigma)$.

5. REDUCE THE SPACE COST TO $\mathcal{O}(n)$

In Algorithm 1, the $\mathcal{O}(n^2)$ space is used for the two tables a and b to compute the values of $\xi(j, i)$ and $\eta(j, i)$ efficiently. It can be observed that when a particular row i of the table a or b is to compute, no rows before the previous row $i-1$ are required. It can also be seen that when a particular row i of the values of $\xi(j, i)$ is to compute, only the column i of the table b is required. When a particular row i of the values of $\eta(j, i)$ is to compute, only the row i and the column i of the table a are required. Thus, only one column of b and one row and one column of a need to be kept in memory at a time. Thus, we need only $\mathcal{O}(n)$ entries to compute $\xi(j, i)$ and $\eta(j, i)$ efficiently.

Algorithm 5: $md(X, Y)$

Input: Two strings X, Y of the same length n.
Output: The mutation distance $md(X, Y)$.
1 $f(0) \leftarrow 0$;
2 **for** $j=1$ **to** n **do** $p(j), q(j), r(j) \leftarrow 0$;
3 **for** $i=1$ **to** n **do**
4 $\quad$ **for** $j=n$ **downto** 1 **do if** $z_j = y_i$ **then** $r(j) \leftarrow r(j-1)+1$:
5 $\quad$ **for** $j=i$ **downto** 1 **do**
6 $\quad\quad$ **if** $x_j = y_i$ **then** $p(j) \leftarrow p(j-1)+1$;
7 $\quad\quad$ **if** $x_j = y_i$ **then** $q(j) \leftarrow q(j-1)+1$;
8 $\quad$ **end**
9 $\quad$ **if** $x_i = y_j$ **then** $f(i) \leftarrow f(i-1)$;
10 $\quad$ **else** $f(i) \leftarrow \infty$;
11 $\quad$ **for** $j=i$ **downto** 1 **do**
12 $\quad\quad$ **if** $f(i) > f(j-1)+1$ **and** $r(n-j+1) \geq i-j+1$ **then** $f(i) \leftarrow f(j-1)+1$;
13 $\quad$ **end**
14 $\quad f(i) \leftarrow \min\{f(i), \gamma(i)\}$;
15 **end**
16 **return** $f(n)$

In the following improved algorithm, the one-dimensional tables p and q are used to store the row i and the column i of the table a, respectively, and the one-dimensional table r is used to store the column i of the table b. The new algorithm can be described as follows.

In the algorithm above, the row i of the table r is computed downward by using the row $i-1$ of the table b, in line 4. The row i and the column i of the table a are computed downward by using the row $i-1$ and the column $i-1$ of the table a, in line 5–8. Then, $\xi(j, i)$ and then $\beta(i)$ are computed in line 12. In line 14, $\gamma(i)$ can be computed by Algorithm 4, using the filter technique, in which $\eta(j, i)$ can be computed by using arrays p and q as follows.

Algorithm 6: $\eta(j, i)$

```
1 for k=j+1 to i do
2 |   if p(j+i-k) ≥ i-k+1   and   q(k-1) ≥ k-j then return k;
3 end
4 return 0;
```

Its time cost is still $\mathcal{O}(i-j)$.

Theorem 4.
Algorithm 5 computes the mutation distance of two equal-length strings in $\mathcal{O}(n^2)$ average time and $\mathcal{O}(n)$ space.

Proof.
The correctness of Algorithm 5 can be guaranteed by Algorithm 1 and 4, since it is a combination of the two algorithms. The computation of $\alpha(i)$ in lines 9–10 costs $\mathcal{O}(1)$ time. The computation of $\beta(i)$ in lines 11–13 costs $\mathcal{O}(i)$ time. It follows that these two parts of Algorithm 5 cost a total of $\mathcal{O}(n^2)$ time in the worst case. It follows from Theorem 3 that the average time cost to compute $\gamma(i)$ for all $1 \leq i \leq n$ can be bounded above by $\mathcal{O}(n^2)$. Therefore, the average time cost of Algorithm 5 can be bounded above by $\mathcal{O}(n^2)$.

The space cost is $\mathcal{O}(n)$ in Algorithm 5, since only four one-dimensional arrays *p, q, r,* and *f* are used.

The proof is completed. ■

6. CONCLUSION

We have presented a linear space algorithm to compute the mutation distance between two strings of the same length under nonoverlapping inversions and transpositions. The recursive formula for this purpose is very simple. The space cost can be reduced from $\mathcal{Q}(n^2)$ in the initial formula to $\mathcal{O}(n)$, and the time cost can be reduced to $\mathcal{O}(n^2)$ in average by using a filter algorithm.

The length of the longest common suffix of two substrings, $lcs(j, i, X, Y)$, plays an important role in our new algorithm. If some suitable data structures such as suffix array or suffix tree are applied, then $lcs(j, i, X, Y)$ can be computed in a constant time, with $\mathcal{O}(n)$ preprocessing time, using $\mathcal{O}(n)$ space. However, in this case, the computation of $\gamma(i)$ remains the bottleneck of the time cost, and thus, the overall complexities are unchanged. It is not clear to us that if such data structures are used, the time cost of $\gamma(i)$ can be reduced further. This leaves room for further exploration.

APPENDIX

Appendix A

In this section, we prove Theorem 1.

The theorem can be proved by induction on n, the size of the input string X. The theorem is trivially true for the cases of $n \leq 1$. Suppose the theorem is quite true when the size of the input string X is less than n. We now show that when the size of the input string X is n, the theorem is also true.

Let

$$f(n)=md(X, Y), \quad l=\min\begin{cases}\alpha(n)\\ \beta(n)\\ \gamma(n)\end{cases}$$

and

$$\begin{cases} C(i)=\{j|1\le j\le i, \xi(j, i)>0\}\\ D(i)=\{j|1\le j<i, \eta(j, i)>0\}\end{cases}$$

There are two cases to be distinguished.

(1) $f(n)<\infty$.

In this case, X can be converted into Y by a set of nonoverlapping mutation operations. For any feasible mutation operation set $\Theta=\{O_1, \cdots, O_p\}$, we can prove $|\Theta|=p\ge l$.

Let the mutation range of $\boldsymbol{O_t}$ be $[j_t, i_t]$, $1\le t\le p$. Since the mutation operations in Θ are mutually nonoverlapping, we can assume $1<i_1<i_2<\cdots<i_p$. It is clear that if the t mutation operations $O_1, \cdots, O_t$ are consecutively applied on X, then $X[1, i_t]$ can be converted to $Y[1, i_t]$ successively.

There are also two subcases to be distinguished.

(1.1) $i_p<n$.

It is clear that in this case, $x_n=y_n$. It follows that $g(n-1)=p\ge f(n-1)=\alpha(n)$. It follows from $\alpha(n)\ge l$ that $p\ge l$.

(1.2) $i_p=n$.

(1.2.1) If $O_p=\theta_{j_p, n}$, then $j_p\in C(n)$. It follows from Equation (10) that $\beta(n)\le f(j_p-1)+1$. On the one hand, it follows from $i_{p-1}\le j_p-1<j_p$ and $f(j_p-1)<\infty$ that $f(j_p-1)=p-1\ge\beta(n)-1$. It follows that $p\ge\beta(n)\ge l$.

(1.2.2) If $O_p=\tau_{j_p, n, k_p}$, for $j_p<k_p\le n$, then $j_p\in D(n)$. It follows from Equation (11) that $\gamma(n)\le f(j_p-1)+1$. On the other hand, it follows from $i_{p-1}\le j_p-1<j_p$ and $f(j_p-1)<\infty$ that $f(j_p-1)=p-1\ge\gamma(n)-1$. It follows that $p\ge\gamma(n)\ge l$.

Especially, for any $\Theta\in Md(X, Y)$, $|\Theta|\ge l$. It follows that

$$f(n)\ge\min\begin{cases}\alpha(n)\\ \beta(n)\\ \gamma(n)\end{cases} \tag{18}$$

On the contrary, if $l=\alpha(n)$, then $x_n=y_n$. Let $\Theta\in Md(X[1, n-1], Y[1, n-1])$. It is clear that $\Theta(X)=Y$. It follows that $f(n)\le|\Theta|=f(n-1)=l$.

In the case of $l=\beta(n)=f(j-1)+1$, and $j\in C(n)$, $1\le j<n$, let $\Theta\prime\in Md(X[1, j-1], Y[1, j-1])$, and $\Theta=\Theta\prime\bigcup\theta_{j, n}$. It is clear that $\Theta(X)=Y$. It follows that $f(n)\le|\Theta|=f(j-1)+1=\beta(n)=l$.

Similarly, in the case of $l=\gamma(n)=f(j-1)+1$, and $j\in D(n)$, $1\le j<k\le n$, let $\Theta\prime\in Md(X[1, j-1]$, $Y[1, j-1])$, and $\Theta=\Theta\prime\bigcup\tau_{j, n, k}$. It is clear that $\Theta(X)=Y$. It follows that $f(n)\le|\Theta|=f(j-1)+1=\gamma(j, n)=l$.

Finally, we have

$$f(n)\le\min\begin{cases}\alpha(n)\\ \beta(n)\\ \gamma(n)\end{cases} \tag{19}$$

It follows from Equations (18) and (19) that

$$f(n)=\min\begin{cases}\alpha(n)\\ \beta(n)\\ \gamma(n)\end{cases} \tag{20}$$

(2) $f(n)=\infty$.

In this case, X cannot be converted into Y by any set of nonoverlapping mutation operations. It is readily seen that in this case, $\alpha(n)=\infty$, $\beta(n)=\infty$, and $\gamma(n)=\infty$. It follows that

$$f(n)=\infty=\min\begin{cases}\alpha(n)\\ \beta(n)\\ \gamma(n)\end{cases} \tag{21}$$

The proof is completed.

Appendix B

In this section, we prove Lemma 2.

Let $k_1=ind(x_{j-1})$ and $k_2=ind(y_{j-1})$.

(1) In the case of $x_{j-1}=y_{j-1}$, we have $k_1=k_2$ and thus $g(j-1,i,k_1)=g(j,i,k_1)$. It follows that $h(j-1,i)=h(j,i)$.

(2) In the case of $x_{j-1}\neq y_{j-1}$, we have $k_1\neq k_2$ and thus

$$g(j-1,i,k)=\begin{cases}g(j,i,k_1)+1 & \text{if } k=k_1\\ g(j,i,k_2)-1 & \text{if} k=k_2\\ g(j,i,k) & \text{if} k\neq k_1 \text{ and } k\neq k_2\end{cases} \tag{22}$$

It follows that

$$\begin{aligned}h(j-1,i)&=\sum_{k=1}^{\sigma}|g(j-1,i,k)|\\ &=\sum_{1\le k\le\sigma,\,k\neq k_1,\,k_2}|g(j-1,i,k)|+|g(j-1,i,k_1)|+|g(j-1,i,k_2)|\\ &=\sum_{1\le k\le\sigma,\,k\neq k_1,\,k_2}|g(j,i,k)|+|g(j,i,k_1)+1|+|g(j,i,k_2)-1|\\ &=\sum_{1\le k\le\sigma}|g(j,i,k)|-\delta_1+\delta_2\\ &=h(j,i)-\delta_1+\delta_2\end{aligned}$$

The proof is completed.

Appendix C

In this section, we prove Theorem 3.

It follows from formula (14) that for each character $c_t\in\Sigma=\{c_1,\cdots,c_\sigma\}$, the number of occurrences of $\boldsymbol{c_t}$ in $X[j,i]$ and $Y[j,i]$ can be written as $g_x(j,i,t)$ and $g_x(j,i,t)$, respectively. It is clear that $X[j,i]$ is a permutation $Y[j,i]$ if and only if $g_x(j,i,t)=g_y(j,i,t)$, $1\le t\le\sigma$. For simplicity, denote $m=i-j+1$, and $s_t=g_y(j,i,t)$ for fixed (j,i), $1\le j\le i\le n$, and $1\le t\le\sigma$. It follows from Lemma 2 that the probability that $X[j,i]$ is a permutation $Y[j,i]$ is exactly

$$\Pr\{j\in P(i)\}=\frac{\binom{m}{s_1}\binom{m-s_1}{s_2}\binom{m-s_1-s_2}{s_3}\cdots\binom{s_\sigma}{s_\sigma}}{\sigma^m} \tag{23}$$

Let $k=\lfloor m/\sigma\rfloor$ and, without loss of generality, we assume that σ divides m. It is clear that the probability given in Equation (23) is maximized when $s_t=k$ for all $1\le t\le\sigma$. It follows that

$$\Pr\{j\in P(i)\}\le\frac{\binom{m}{k}\binom{m-k}{k}\binom{m-2k}{k}\cdots\binom{k}{k}}{\sigma^m}=\frac{m!}{(k!)^\sigma\sigma^m}$$

It follows from Stirling's approximation for both $m!$ and $k!$ that

$$\frac{m!}{(k!)^\sigma\sigma^m}=\Theta\left(\frac{\sqrt{2\pi m}(m/e)^m}{\left(\sqrt{2\pi(m/\sigma)}(m/(e\sigma))^{m/\sigma}\right)^\sigma\sigma^m}\right)=\Theta\left(\frac{\sqrt{2\pi m}}{\left(\sqrt{2\pi(m/\sigma)}\right)^\sigma}\right)$$

It is clear that $\sqrt{2\pi}/(\sqrt{2\pi})^{\sigma} \leq 1$, and thus

$$\Theta\left(\frac{\sqrt{m}}{\left(\sqrt{m/\sigma}\right)^{\sigma}}\right) = \Theta\left(\frac{\sigma^{\sigma/2}}{m^{(\sigma-1)/2}}\right)$$

It follows that

$$\Pr\{j \in P(i)\} \leq \frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-1)/2}} \tag{24}$$

The time cost to check for $\eta(j, i) > 0$ is $\mathcal{O}(i-j+1)$. Therefore, $T_{\beta}(n)$, the average time to compute $\gamma(i)$, $1 \leq i \leq n$ is bounded above by

$$\begin{aligned} T_{\beta}(n) &\leq \sum_{1\leq j\leq i\leq n} \Pr\{j \in P(i)\} \cdot \mathcal{O}(i-j+1) \\ &= \sum_{1\leq j\leq i\leq n} \mathcal{O}\left(\frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-1)/2}} \cdot (i-j+1)\right) \\ &= \sum_{1\leq j\leq i\leq n} \mathcal{O}\left(\frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-3)/2}}\right) \end{aligned}$$

It follows from the assumption that $\sigma \geq 3$ is a constant that $\frac{\sigma^{\sigma/2}}{(i-j+1)^{(\sigma-3)/2}} = \mathcal{O}(1)$. It follows that

$$T_{\beta}(n) = \sum_{1\leq j\leq i\leq n} \mathcal{O}(1) = \mathcal{O}(n^2)$$

The proof is completed.

AUTHOR DISCLOSURE STATEMENT

No competing financial interests exist.

REFERENCES

Apostolico, A., Guerra, C., and Landau, G.M. 2016. Sequence similarity measures based on bounded hamming distance. *Theor. Comput. Sci.* 638, 79–90.

Cantone, D., Cristofaro, S., and Faro, S. 2013. Efficient string-matching allowing for non-overlapping inversions. *Theor. Comput. Sci.* 483, 85–95.

Chen, Z., and Gao, Y. 2004. A space-efficient algorithm for sequence alignment with inversions and reversals. *Theor. Comput. Sci.* 325, 361–372.

Cho, D., Han, Y., and Kim, H. 2015. Alignment with non-overlapping inversions and translocations on two strings. *Theor. Comput. Sci.* 575, 90–101.

Gao, Y., Wu, J., and Niewiadomski, R. 2003. A space efficient algorithm for sequence alignment with inversions, 57–67. In *Proc. COCOON 2003, volume 2697 of Lecture Notes in Computer Science*. Springer-Verlag, London.

Grabowski, S., Faro, S., and Giaquinta, E. 2011. String matching with inversions and translocations in linear average time (most of the time). *Inf. Process. Lett.* 111, 516–520.

Huang, S., Yang, C., and Ta, T. 2014. Approximate string matching problem under non-overlapping inversion distance, 38–46. In *Proc. 2014 International Computer Symposium, volume 1 of ICS'14*. IOS Press, Amsterdam.

Huang, Y., and Lu, C. 2010. Sorting by reversals, generalized transpositions, and translocations using permutation groups. *J. Comput. Biol.* 17, 685–705.

Li, S., and Ng, Y. 2009. On protein structure alignment under distance constraint, 65–76. In *Proc. ISAAC 2009, volume 5878 of Lecture Notes in Computer Science*. Springer-Verlag, London.

Lipsky, O., Porat, B., and Porat, E. 2007. Approximate string matching with swap and mismatch, 869–880. In *Proc. ISAAC 2007, volume 4835 of Lecture Notes in Computer Science*. Springer-Verlag, London.

Oliver-Bonet, M., and Navarro, J. 2002. Aneuploid and unbalanced sperm in two translocation carriers: Evaluation of the genetic risk. *Mol. Hum. Reprod.* 8, 958–963.

Sakai, Y. 2011. A new algorithm for the characteristic string problem under loose similarity criteria, 663–672. In *Proc. ISAAC 2011, volume 7074 of Lecture Notes in Computer Science*. Springer-Verlag, London.

Schoniger, M., and Waterman, M. 1992. A local algorithm for dna sequence alignment with inversions. *Bull. Math. Biol.* 54, 521–536.

Ta, T., Lin, C., and Lu, C. 2016. An efficient algorithm for computing non-overlapping inversion and transposition distance. *Inf. Process. Lett.* 116, 744–749.

Vellozo, A., Alves, C., and do Lago, A. 2006. Alignment with non-overlapping inversions in $o(n^3)$ time, 186–196. In *Proc. WABI 2006, volume 4175 of Lecture Notes in Computer Science*. Springer-Verlag, London.

Address correspondence to:
Dr. Lei Wang
Facebook, Inc.
Menlo Park, CA 94052

E-mail: soft139@qq.com